



MODEL
CONTEXT
PROTOCOL



LEARN



BUILD



INTEGRATE



EXTEND

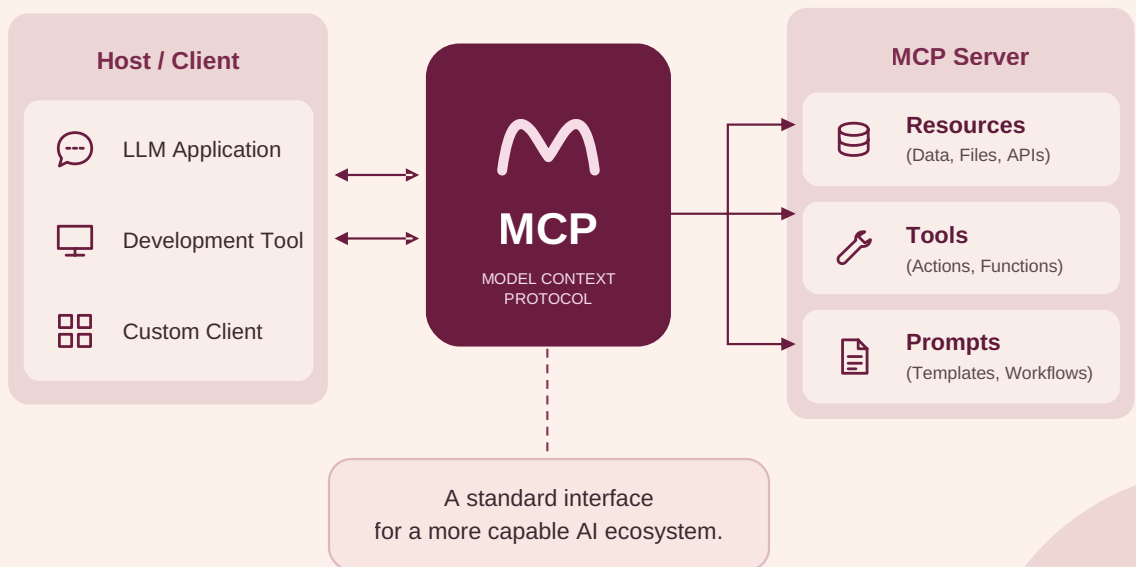
AI ENGINEERING ROADMAP · STUDY NOTES

BUILD
CONNECT
EXTEND

CHAPTER 01

The MCP Trailer: Why This Protocol Matters

A real newsletter-automation project shows why Model Context Protocol changes how AI engineers build.



Premium Revision Notes

CONCEPTS

Clear Explanations

IMPLEMENTATION

Practical Code

DIAGRAMS

Visual Learning

INTERVIEW PREP

Key Questions

SMARTER AI.
BIGGER POSSIBILITIES.

Chapter 1: The MCP Trailer

This chapter opens the MCP study series with a trailer: a complete, real engineering project built end-to-end using MCP, meant to show — not just tell — why the protocol matters. The series itself unfolds across three arcs following this trailer: **The Why** (why MCP exists), **The What** (how MCP works architecturally, across two parts), and **The How** (practical implementation, across four parts).

REAL WORLD INSIGHT

The demonstration is built around a real internal tool: an AI-generated weekly newsletter, built to solve a genuine problem — keeping up with the breakneck pace of AI development is hard even for full-time practitioners.

1. The Problem: Staying Current in AI

AI moves fast enough that a six-month learning roadmap can become partially obsolete within a month or two, a problem working engineers face as much as students. One habit that helps is subscribing to multiple AI newsletters and reading at least one daily. But *producing* a good newsletter is itself expensive: it demands daily research, careful writing, and design, none of which scale well against a limited amount of time.

IMPORTANT

The core idea of this chapter is not "AI can write a newsletter." It is that a multi-step, multi-source content pipeline — research, drafting, design — can be automated by connecting one AI application to many tools through a single standardized protocol: MCP.

2. Designing the Newsletter Structure

Before automating anything, the project defined what the final product should look like. Studying existing AI newsletters, a nine-section structure was settled on:

#	Section	Purpose
1	Introduction	Short paragraph building curiosity about what follows
2	Big Story of the Week	The single most significant AI news item
3	Quick Updates	3–5 smaller but noteworthy news items
4	Top Research Papers	Notable papers with summaries and links
5	GitHub Repos	Trending open-source repositories
6	Learning Corner	A short, 5-minute-readable tutorial
7	Top AI Products	Newly launched products, in brief
8	Top Tweets	What leading AI voices are discussing
9	Closing Notes	Summary plus a teaser for next week's edition

Quick Revision

- The newsletter has 9 fixed sections, from Introduction to Closing Notes.
- The structure was designed manually first, before any automation.
- This is presented as a good starting structure, not a claimed "best" one.

3. Defining the Automation Process

With the target structure fixed, the next step was to define exactly how an AI system would produce it. The entire creation process was broken into three stages: **Research**, where the assistant gathers material from several sources; **Editing**, where that material is assembled into one final draft matching the newsletter format; and **Designing**, where the draft is converted into a styled, sendable HTML email.

Choosing the Tools

Claude was chosen as the main assistant because MCP originates from Anthropic, and at the time of the project, Claude had the strongest MCP support compared to other chatbots — ChatGPT, for example, lacked comparable integrations. Around six or seven external tools were connected to Claude for this project: Google Drive, Gmail, web search, GitHub, arXiv, Product Hunt, and Twitter/X — all wired in through MCP.

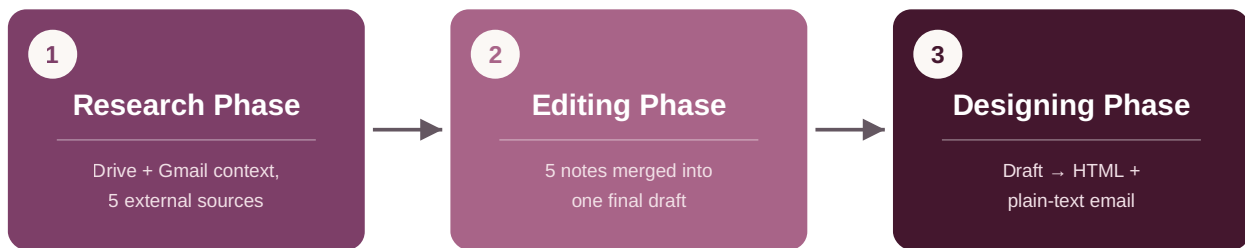


Figure 1.1 — The three-stage newsletter creation pipeline: Research, Editing, Designing.

INTERVIEW TIP

Be ready to explain *why* a standardized protocol like MCP is preferable to writing a custom integration for every tool. This chapter only demonstrates the benefit practically; the upcoming Why and What parts of this series cover the architectural reasoning in depth.

🔄 Quick Revision

- Three stages: Research → Editing → Designing.
- Claude was chosen for its strong native MCP support.
- Roughly 6–7 external tools were connected via MCP: Drive, Gmail, web search, GitHub, arXiv, Product Hunt, Twitter/X.

4. Stage 1 — Research Phase

The research stage itself has two parts: first, deciding *what* to research, and second, actually conducting that research.

4.1 Deciding What to Research

Before searching anywhere external, Claude was instructed to first read context from three internal sources stored in Google Drive and Gmail:

- **Content Ideas** — a Drive document listing candidate topics the newsletter could cover.
- **Performance Data** — a Drive document with dummy metrics (open rate, click rate, average read time) modeled on what a tool like Mailchimp would report for past sends.
- **Feedback Emails** — recent reader feedback pulled directly from Gmail.

Since this was the very first edition, there was no real performance history yet — the performance-data file used placeholder values. The intent was to build the habit into the workflow now, so that future editions genuinely benefit from real audience data once it exists.

BEST PRACTICE

Feed the agent your own first-party signals (past performance, direct feedback, a running list of ideas) before sending it out to research the open web. Grounding the research phase in real context produces more targeted results than a cold, generic search.

4.2 Conducting the Research

Once Claude derived focus areas from the three internal sources (for example: AI in healthcare and finance, OpenAI vs. Anthropic competition, Llama 3.2 and AI hardware, open-source LLMs), it was sent out to five external sources in parallel, each producing its own dedicated Markdown research file saved to the local file system:

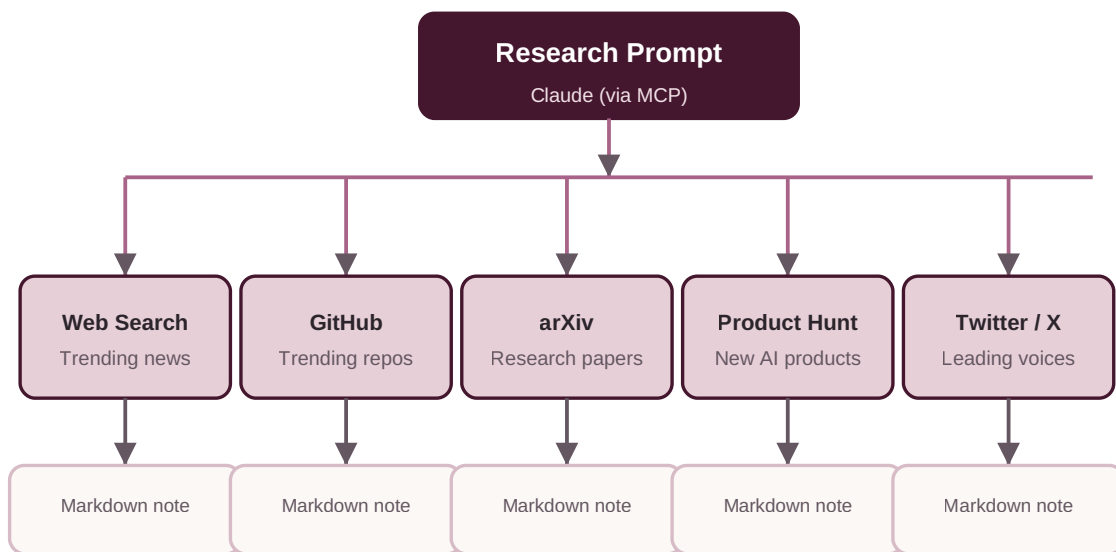


Figure 1.2 — One research prompt fans out into five parallel research branches, each producing its own note.

COMMON MISTAKE

Assuming an agent must complete research sources strictly one after another. In practice, Claude was observed starting work on one branch (for example, GitHub) while a previous branch (for example, arXiv) was still finishing, conducting the five research branches with meaningful parallelism.

RESULTING FILES

The research stage ends with five separate Markdown files on disk, each scoped to one source: web search results (the "Big Story" and "Quick Updates" material), arXiv paper summaries, trending GitHub repositories, Product Hunt launches, and popular AI-related tweets.

Quick Revision

- Research is grounded first in Drive content ideas, Drive performance data, and Gmail feedback.
- Five parallel research branches: web search, GitHub, arXiv, Product Hunt, Twitter/X.
- Each branch produces its own standalone Markdown file.
- Claude can work on multiple research branches with observable parallelism.

5. Stage 2 — Editing Phase

With five research files sitting on the desktop, the second prompt instructs Claude to act as an "AI newsletter assembly agent." It reads all five research files, then goes back to Google Drive to re-read the sample newsletter template that defines the nine-section format. Using both inputs, it merges the scattered research into one coherent Markdown draft, populating every section, smoothing transitions between them, and keeping an engaging editorial tone throughout. The resulting draft is saved back into the same project folder on the desktop.

IMPORTANT

The editing prompt does not re-do any research. It strictly recombines material that already exists in the five source files against a fixed template — keeping its job narrow and predictable.

Quick Revision

- Editing reads the 5 research files plus the Drive sample template.
- Output is a single consolidated Markdown draft, not new research.
- The draft is saved back to the same desktop project folder.

6. Stage 3 — Designing Phase

The final prompt asks Claude to act as an "email template builder agent." It reads the finalized Markdown draft from the desktop and converts it into a production-ready HTML email, following a set of layout, styling, content-mapping, and deliverability requirements supplied in the prompt, along with an example HTML markup pattern to follow. Two outputs are generated: an HTML version and a plain-text version.

BEST PRACTICE

Always produce a plain-text fallback alongside an HTML email. Some email clients mark heavily styled HTML mail as spam or fail to render it; a plain-text version ensures the content still reaches the reader.

Both files are saved back to the same project folder. At that point, the only remaining manual step is uploading the HTML file to an email tool such as Mailchimp and sending it to the subscriber list.

🔗 Quick Revision

- Designing converts the final Markdown draft into a styled HTML email.
- A plain-text version is generated alongside the HTML version for deliverability.
- Sending itself (e.g. via Mailchimp) remains a manual, external step.

7. How Little Code MCP Actually Requires

A natural question after seeing this many tools connected to one assistant is how much integration code was required. The answer, in this project, is effectively none in the traditional sense: no custom API calls or per-tool functions were written. Instead, each tool was added to Claude Desktop by editing a single configuration file (`claude_desktop_config.json`) with one JSON entry per MCP server.

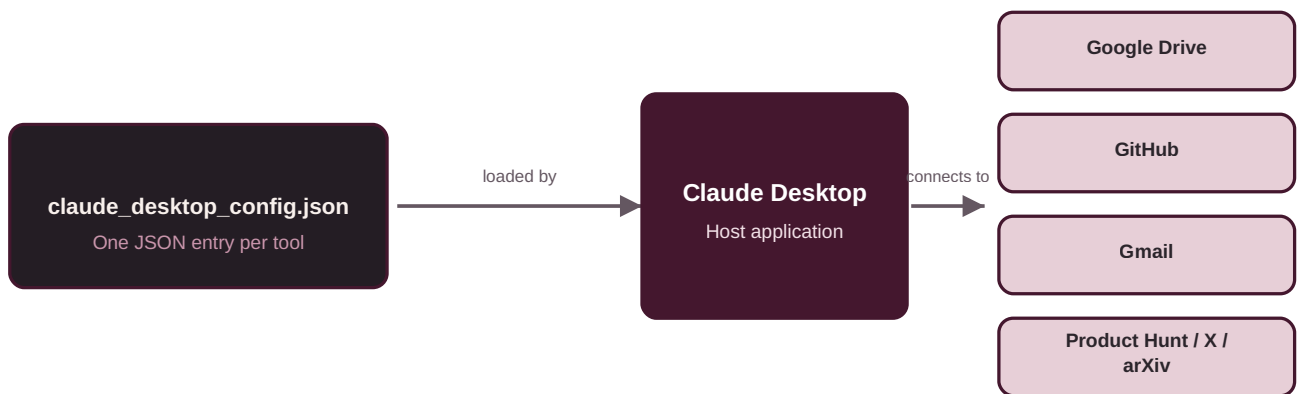


Figure 1.3 — Tool integration is configuration, not custom code: one JSON entry per MCP server.

```
# Example shape of an MCP server entry (secrets replaced with placeholders)
{
  "mcpServers": {
    "twitter": {
      "command": "npx",
      "args": ["-y", "<TWITTER_MCP_PACKAGE>"],
      "env": {
        "TWITTER_API_KEY": "<YOUR_API_KEY>"
      }
    }
  }
}
```

REAL WORLD INSIGHT

If a tool's underlying MCP server changes internally, the config on the client side generally does not need to change. The server implementation handles the details; the client only needs a stable configuration entry pointing at it.

COMMON MISTAKE

Assuming connecting many tools to an LLM requires writing a comparable amount of glue code for each one. MCP's value proposition is precisely that this integration burden moves into a standardized configuration step instead.

🔄 Quick Revision

- Adding an MCP server to Claude Desktop is a config change, not custom code.
- One JSON entry per tool in `claude_desktop_config.json`.
- Server-side changes to a tool's MCP implementation typically don't require client-side code changes.

8. Where This Series Goes Next

This trailer intentionally leaves the internal mechanics of MCP unexplained. Terms like *host*, *client*, and *server* were used loosely here to describe how Claude Desktop reached external tools such as a calendar or Google Drive. The series proper unpacks this precisely, across three arcs:

Part	Focus
The Why	Why MCP exists: the integration problem it solves and the motivation behind the protocol
The What (2 parts)	Architecture: hosts, clients, and servers, and exactly how they interact
The How (4 parts)	Hands-on implementation: building MCP servers and clients, solving real use cases in code

Key Takeaways

- MCP allowed one assistant (Claude) to be wired into roughly 6–7 external tools — Drive, Gmail, web search, GitHub, arXiv, Product Hunt, Twitter/X — without writing custom integration code for each.
- The full newsletter pipeline was executed in exactly three prompts, corresponding to three stages: Research, Editing, Designing.
- Research was deliberately grounded in first-party data (content ideas, performance history, reader feedback) before reaching out to external sources.
- Adding a new MCP-connected tool is a configuration change (one JSON entry), not a new integration codebase.
- This chapter is a motivational demonstration; architectural depth and hands-on server/client building are deferred to the Why, What, and How parts of this series.

The 3-Stage Pipeline

Research (gather from 5 sources) → Editing (merge into one draft) → Designing (convert draft to HTML + plain-text email).

Grounding Before Searching

Read Drive content ideas + performance data + Gmail feedback first, then decide what topics to externally research.

Five Research Branches

Web search, GitHub, arXiv, Product Hunt, Twitter/X — each producing an independent Markdown note, run with observable parallelism.

Integration = Configuration

Every MCP tool is added via one entry in `claude_desktop_config.json`; no bespoke API code is required per tool.

Interview Questions

Q1. Why was Claude chosen over other chatbots for this MCP-based project?

Q2. What are the three stages of the newsletter automation pipeline, and what does each stage take as input?

Q3. Why is the research phase grounded in internal Drive/Gmail data before searching external sources?

Q4. Why are both an HTML and a plain-text version of the newsletter generated?

Q5. What kind of "integration code" does connecting a new tool via MCP actually require?

Further Reading

- [Anthropic — Introducing the Model Context Protocol](#)
- [The Why — this series' next part: why MCP exists](#)
- [The What — MCP architecture, hosts, clients and servers \(2 parts\)](#)
- [The How — building MCP servers and clients \(4 parts\)](#)